UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY (FET)
PO BOX 63

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

BUEA, SOUTH WEST REGION

TASK 4: System Modelling and Design.

COURSE TITLE: Internet Programming & Mobile Programming
COURSE INSTRUCTOR: **Dr. Nkemeni Valery**

SUBMITTED BY
GROUP 3 MEMBERS

NAME	MAT
KONEH CATHERINE MBEH	FE23A076
MOFA DIVINE ESIMO	FE23A095
TUBE NDIANG MARY SHARON	FE23A051
AGBORNJA BESONG AKPAK DIVINE	FE23A006
OBIOMA OBI FODJO CAREL	FE23A133

DEPARTMENT OF COMPUTER ENGINEERING

Table of Contents

INTRODUCTION TO SYSTEM MODELLING AND DESIGN

System modelling and design is a fundamental phase in software engineering that focuses on representing the structure, behaviour, architecture, and interactions of a software system before implementation. It provides a visual and technical understanding of how different components of a system operate, communicate, and work together to achieve the intended objectives. Through modelling, developers, analysts, and stakeholders are able to understand the functionality of the proposed system, identify requirements clearly, and simplify the development process. For the Crowd sensed Drive Test Mobile Application for Coverage Prediction, system modelling and design plays an important role in illustrating how mobile network measurements are collected, processed, stored, analysed, and visualized within the application. Since the system is designed as an Android based crowdsensing platform integrated with cloud infrastructure and geographic services, several modelling diagrams are required to explain different aspects of the system architecture and operation.

The system modelling process for this project includes the development of multiple diagrams, each representing a specific perspective of the system. The Context Diagram provides a high-level overview of the entire system and shows how external entities such as users, administrators, Telephony APIs, GPS APIs, and Google Maps APIs interact with the application. The Data Flow Diagram (DFD) explains how data moves through the system, beginning from signal collection on Android smartphones to synchronization, analytics generation, and report production.

The Use Case Diagram illustrates the functional interactions between users, administrators, and the system by identifying the major services and functionalities provided by the application. The Sequence Diagram represents the chronological interaction between system components during important use cases such as user authentication, signal monitoring, synchronization, and report generation. The Class Diagram models the structural design of the application by showing the system classes, attributes, methods, and relationships between objects. In addition, the Deployment Diagram explains the physical architecture of the system by illustrating how software components are deployed across Android smartphones, cloud backend servers, database servers, administrator workstations, and external geographic services such as Google Maps APIs. Together, these modelling diagrams provide a complete understanding of the architecture, communication flow, data processing, and deployment structure of the Crowd sensed Drive Test Mobile Application.

1) CONTEXT DIAGRAM

A Context Diagram is one of the most important diagrams used during system analysis and design. It provides a high-level representation of a system by showing the system as a single process and illustrating how external entities interact with it through the exchange of data. The purpose of this diagram is to define the system boundary clearly and identify the flow of information between the system and its environment. For the Crowdsensed Drive Test Mobile Application, the context diagram is used to represent how users, APIs, servers, databases, and administrators interact with the application. The diagram helps developers and stakeholders understand the major components involved in the system and how information moves between them. It also simplifies communication during the software development process because it presents the overall structure of the system in a clear and organized manner.

The Crowdsensed Drive Test Mobile Application is designed to collect mobile network measurements from smartphone users in order to predict network coverage and generate coverage maps. The application relies on

several external entities such as the Android Telephony API, GPS services, backend servers, databases, and mapping services to function effectively.

Purpose of the Context Diagram

The context diagram serves several important purposes in system analysis and design.

It helps to:

- Represent the entire system as one main process.
- Identify all external entities interacting with the system.
- Show the inputs entering the system.
- Show the outputs produced by the system.
- Define the boundaries of the application.
- Provide a simplified overview of system interactions.

In this project, the context diagram helps explain how the Crowdsensed Drive Test Mobile Application communicates with users, external APIs, backend infrastructure, and administrative components.

1.1 Main System

At the center of the context diagram is the main system known as the:

"Crowdsensed Drive Test Mobile Application"

This application is responsible for:

- Collecting crowdsensed network measurements.
- Monitoring signal quality.
- Capturing GPS coordinates.
- Sending collected data to the backend server.
- Generating analytics and coverage predictions.
- Displaying coverage maps and reports to users.

The application acts as the core processing unit through which all external entities exchange information.

1.1.1 External Entities in the Context Diagram

a) User

The User represents the smartphone user interacting directly with the mobile application. The user plays a major role in the crowd-sensing process because the application depends on user participation to gather network data.

The user performs the following activities:

- Registers and logs into the application.

- Grants' device permissions.
- Monitors signal quality.
- Views generated coverage maps.
- Sends crowdsensed network data.
- Provides feedback to the system.

The user is therefore both a source of input data and a receiver of processed information from the system.

b) Android Telephony API

The Android Telephony API provides cellular network information collected by the smartphone. This API enables the application to monitor mobile network performance and signal quality.

The Android Telephony API provides the following signal parameters:

- RSSI (Received Signal Strength Indicator)
- RSRP (Reference Signal Received Power)
- RSRQ (Reference Signal Received Quality)
- SINR (Signal-to-Interference-plus-Noise Ratio)
- Cell ID
- Network Type

These measurements are essential because they form the core network data used for coverage prediction and signal analysis.

c) GPS / Location Services API

The GPS or Location Services API provides geographical information required for geo-tagging network measurements. This allows the application to associate signal strength values with specific locations.

The GPS API provides:

- Latitude
- Longitude
- Speed
- Altitude
- Device heading

These parameters help the system generate accurate location-based coverage maps.

d) Backend Server

The Backend Server is responsible for processing, analyzing, and synchronizing the crowdsensed data collected from users. It acts as the communication bridge between the mobile application and the database server.

The backend server receives:

- Signal measurements
- GPS coordinates
- Synchronization requests

The backend server returns:

- Coverage maps
- Analytics
- Notifications
- Reports

The backend server also performs data processing and predictive analysis required for network coverage prediction.

e) Database Server

The Database Server stores all important system information and collected network records. It ensures that user data and measurement data are safely maintained for future processing and analysis.

The database stores:

- User data
- Signal records
- Coverage analytics
- Logs

The database supports long-term storage and retrieval of crowdsensed information.

f) Google Maps API

The Google Maps API provides mapping and geographic visualization services for the application. It helps display network coverage data visually on digital maps.

The Google Maps API provides:

- Base maps
- Geographic visualization

This enables users to view signal coverage results in an interactive and understandable format.

g) Administrator

The Administrator manages the overall operation of the system. The administrator is responsible for monitoring system performance and managing analytics and reports.

The administrator performs the following functions:

- Managing users
- Monitoring analytics
- Generating reports
- Managing monitoring tools

The administrator ensures proper system management and supervision.

1.1.2 Data Flows in the Context Diagram

Inputs into the System

The Crowdsensed Drive Test Mobile Application receives several forms of input data from users and external APIs. These inputs include:

- Signal measurements
- GPS coordinates
- User credentials
- Permissions
- Feedback
- Synchronization requests

These inputs are processed by the system to generate analytics and coverage predictions.

Outputs from the System

After processing collected data, the system produces several outputs for users and administrators. These outputs include:

- Coverage maps
- Notifications
- Signal statistics
- Reports
- Analytics dashboards

The outputs help users understand network quality and assist administrators in monitoring system performance.

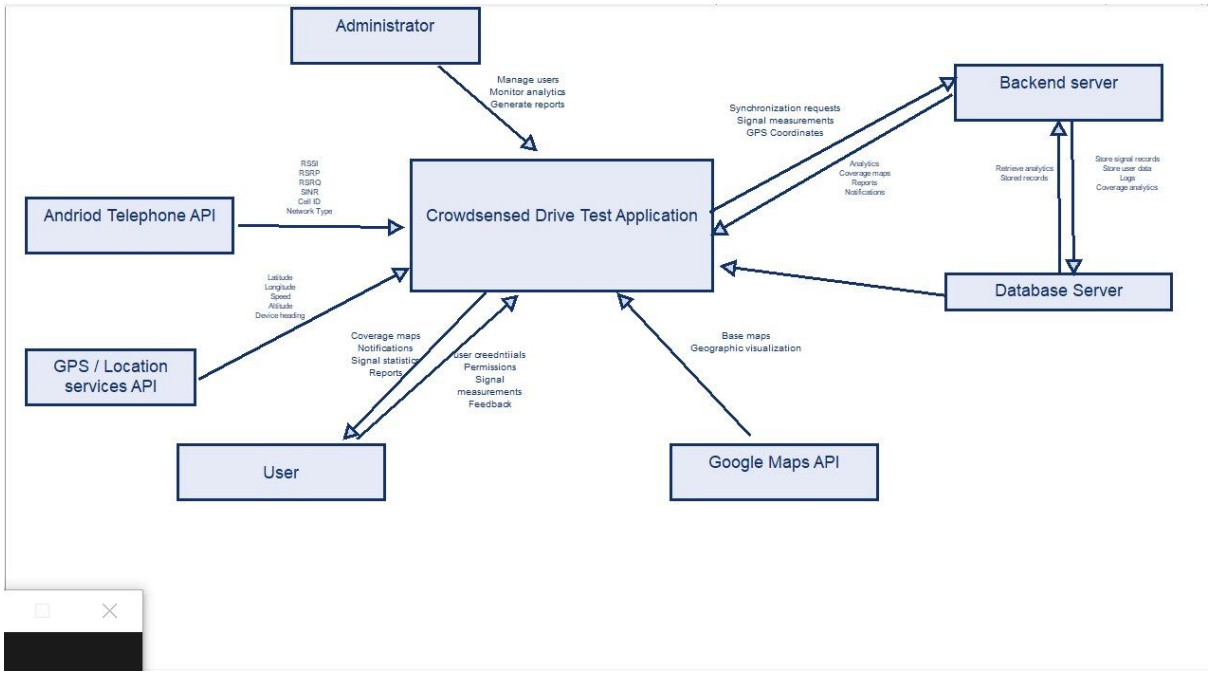

Figure 1: Context Diagram For the system

Description of the Diagram Layout

In the context diagram, the "Crowdsensed Drive Test Mobile Application" is placed at the center because it is the main process being analyzed. Around the central process are the external entities interacting with the system.

The User, Android Telephony API, and GPS API are positioned on one side of the system because they mainly provide input data. The Backend Server, Database Server, Google Maps API, and Administrator are positioned around the remaining sides because they support processing, storage, visualization, and management operations.

Arrows are used to represent data flow between the system and each external entity. Each arrow is labeled according to the type of information exchanged.

Importance of the Context Diagram

The context diagram is important because it provides a simplified but complete overview of the entire system. It allows developers, stakeholders, and users to understand how the system communicates with external entities without focusing on internal implementation details.

For the Crowdsensed Drive Test Mobile Application, the context diagram helps to:

- Clarify system boundaries.
- Identify system dependencies.
- Understand major data flows.
- Improve communication among developers and stakeholders.
- Support further system design activities.

The diagram also serves as the foundation for more detailed system models such as Data Flow Diagrams, Use Case Diagrams, and architectural designs.

2) Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation used to illustrate how data moves within a software system. It explains the movement of information between external entities, processes, data stores, and system outputs. In the Crowd sensed Drive Test Mobile Application, the DFD is used to explain how crowd sensed mobile network measurements are collected from Android smartphones, processed by the system, stored in databases, synchronized with cloud infrastructure, and transformed into meaningful analytics and coverage prediction results. The Data Flow Diagram helps provide a clearer understanding of the operational workflow of the application. It demonstrates how users interact with the system, how signal measurements are captured from the smartphone modem, how GPS coordinates are attached to the measurements, how synchronization with backend services occurs, and how analytics and reports are generated. The DFD for this project is divided into two major levels. The Level 0 DFD presents the entire system as a single process interacting with external entities, while the Level 1 DFD expands the major internal process known as Signal Data Collection into smaller detailed subprocesses.

2.1 Detailed Flow of Data Within the System

External Entities

The system interacts with the following external entities:

1. User

The user interacts with the Android mobile application to:

- Register an account
- Login into the system
- Start monitoring
- View analytics and coverage maps
- Generate reports

2. Admin

The administrator monitors system activities and analytics through the administrator dashboard.

3. Telephony API

The Android Telephony API provides:

- RSSI
- RSRP
- RSRQ
- SINR
- Cell ID
- Network Type

4. GPS API

The GPS API provides:

- Latitude
- Longitude
- Altitude
- Speed
- Timestamp

5. Google Maps API

The Google Maps API provides:

- Geographic rendering services
- Map tiles
- Coverage visualization support

The data flow process of the Crowd sensed Drive Test Mobile Application begins when the user launches the Android mobile application and starts the monitoring process. Once monitoring is activated, the application communicates directly with the Android Telephony API available on the smartphone. Through this interaction, the application retrieves several important cellular network measurements including RSSI, RSRP, RSRQ, SINR, Cell ID, and network type. These measurements represent the current state and quality of the mobile network connection around the user.

At the same time, the application communicates with the GPS API in order to obtain geographical information associated with the signal measurements. The GPS service provides latitude, longitude, altitude, speed, and timestamp information. These coordinates are important because they allow the system to identify the exact location where each signal measurement was collected.

After retrieving both signal and geographical information, the application combines the datasets to create complete geo-referenced measurement records. Each measurement therefore contains both signal strength information and its corresponding geographical coordinates.

The system then validates the collected measurements before storing them. During this validation stage, the application checks for missing values, duplicated records, abnormal signal ranges, and corrupted measurements. Invalid records are filtered out while valid measurements continue through the processing flow. This validation process improves the reliability and quality of the crowd sensed data.

Once validation is completed, the measurements are temporarily stored inside the SQLite database located on the Android smartphone. Local storage is necessary because the system must continue operating even when internet connectivity becomes unstable or unavailable. The SQLite database therefore acts as an offline buffer that prevents loss of crowd sensed measurements.

Whenever internet connectivity becomes available, the Synchronization Module retrieves the buffered measurements from the local SQLite database and prepares them for transmission to the cloud backend server. Before transmission, the measurements are converted into JSON format and encrypted using HTTPS and TLS

1.2 security protocols. Encryption ensures that the transmitted measurements remain secure during communication.

The encrypted measurements are then synchronized with the Cloud Backend Server through RESTful APIs. The backend server receives the uploaded records through the REST API service and verifies user identity using the Authentication Service. After successful authentication, the synchronization service processes the uploaded measurements and forwards them for permanent storage.

The validated measurements are stored inside the Measurement Database located on the Database Server. System activities such as synchronization attempts, upload status, and system errors are also recorded inside the Logs Database for monitoring and debugging purposes.

After storage, the Analytics Engine processes the crowd sensed measurements in order to generate coverage heatmaps, weak signal zone detection, and coverage prediction analytics. The processed analytics are then integrated with the Google Maps API to visualize the results geographically. Coverage maps and prediction overlays are displayed on digital maps to help users and administrators understand mobile network performance across different geographical areas.

Finally, the generated analytics and reports are made available to users and administrators through the Android application and administrator dashboard. Administrators can monitor synchronization activities, analyse coverage statistics, and generate PDF or CSV reports using the processed crowd sensed data.

2.2 Level 0 Data Flow Diagram

The Level 0 Data Flow Diagram, also known as the Context Diagram, provides a high-level representation of the entire system. At this level, the whole application is represented as a single process interacting with external entities. The purpose of the Level 0 DFD is to define the system boundary and illustrate the major data interactions occurring between the application and external actors.

The main external entities interacting with the system include the User, Administrator, Telephony API, GPS API, and Google Maps API. Users interact with the Android application to perform registration, login, signal monitoring, synchronization, and report viewing activities. The Telephony API provides cellular network measurements while the GPS API provides location-based information. The Google Maps API supports map rendering and geographic visualization.

The major data flows represented in the Level 0 DFD include login credentials, crowd sensed signal measurements, GPS coordinates, synchronization data, coverage analytics, reports, and map visualizations.

The major data flows include:

- Login credentials
- Signal measurements
- GPS coordinates
- Coverage analytics
- Synchronization data

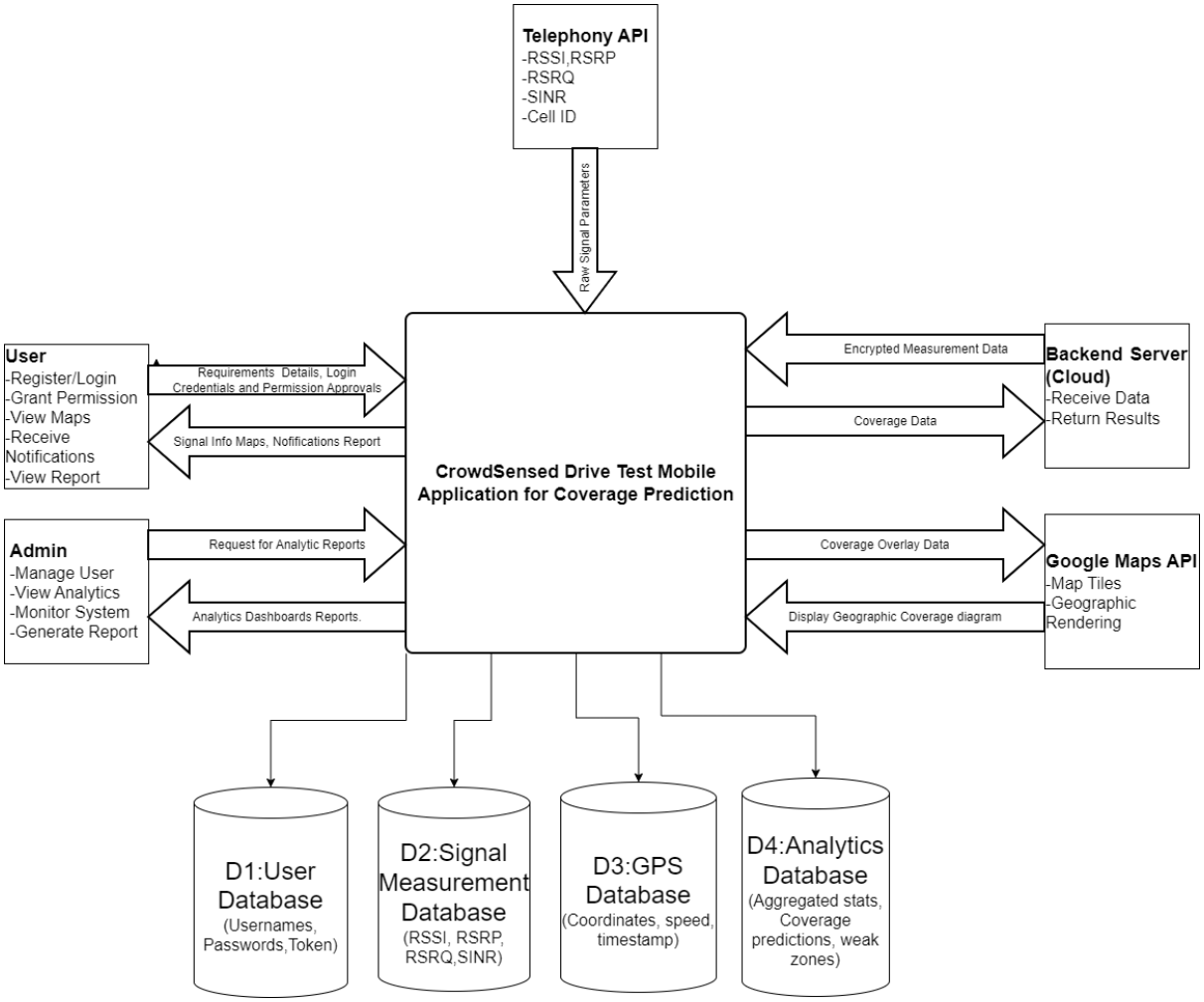

Figure 2.1: Level 0 Data Flow Diagram (Context Diagram)

2.3 Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram expands the major internal process known as Signal Data Collection into smaller subprocesses in order to provide a more detailed explanation of the internal workflow of the system.

The first subprocess involves reading signal measurements from the Android Telephony API. During this stage, the system retrieves RSSI, RSRP, RSRQ, SINR, Cell ID, and network type information from the smartphone modem.

The collected measurements are then passed through a validation process where the system checks for missing values, duplicated records, corrupted measurements, and abnormal signal ranges. Measurements that pass validation continue through the processing workflow.

After validation, the system retrieves GPS coordinates from the GPS API and attaches them to the validated signal measurements. This creates geo-referenced crowd sensed records that contain both signal information and geographical location.

The generated records are then stored locally inside the SQLite database available on the Android smartphone. Local storage supports offline operation and prevents data loss when internet connectivity becomes unstable.

When connectivity becomes available, the synchronization process begins. The records are converted into JSON format, encrypted using HTTPS and TLS 1.2 security, and transmitted securely to the Cloud Backend Server through RESTful APIs.

The backend synchronization service receives the uploaded measurements and forwards them to the Analytics Engine for processing. The Analytics Engine generates coverage heatmaps, weak signal detection results, prediction analytics, and other coverage statistics. These analytics are stored inside the Analytics Database and integrated with Google Maps services for geographic visualization.

The Level 1 DFD therefore provides a detailed representation of how crowd sensed measurements move through the system from data collection to analytics generation.

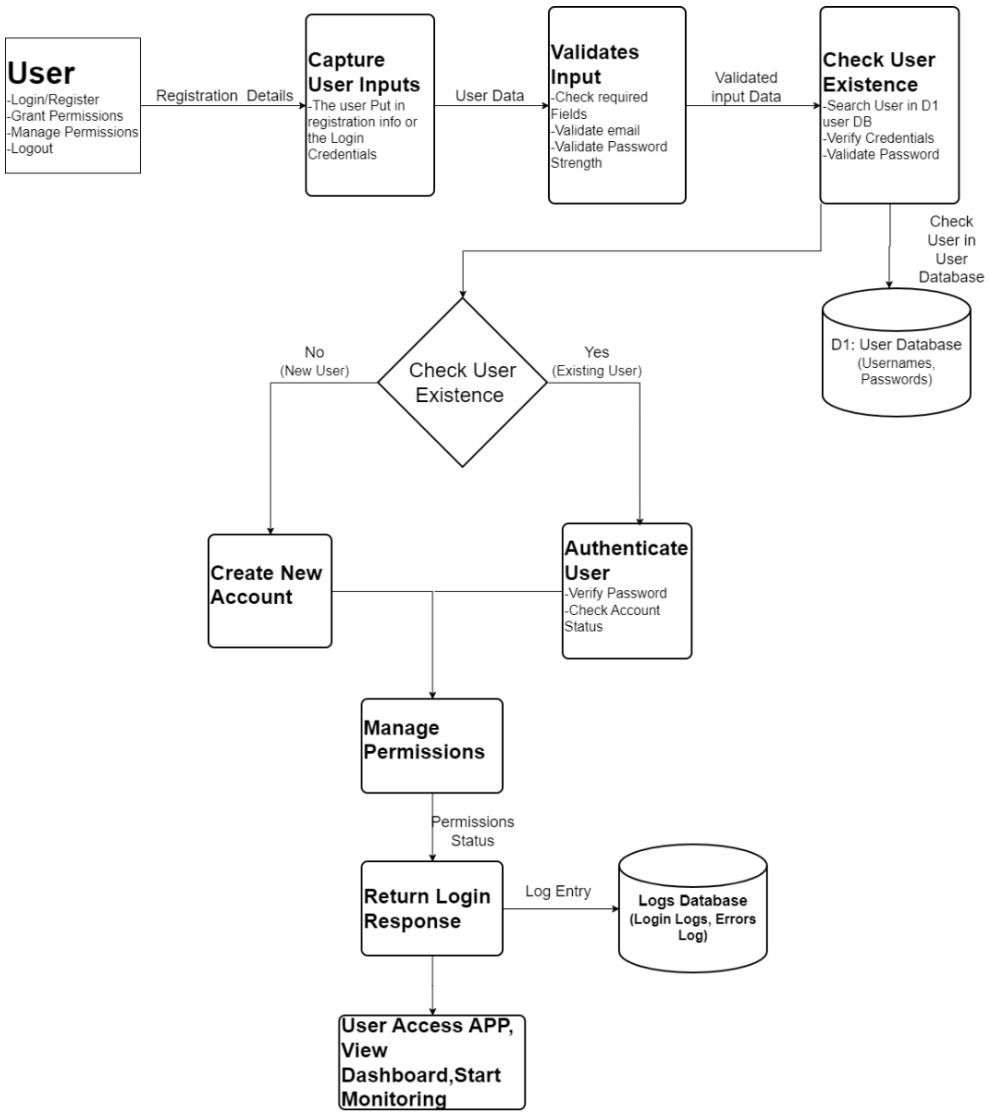

Figure 2.2: Level 1 Data Flow Diagram for user login

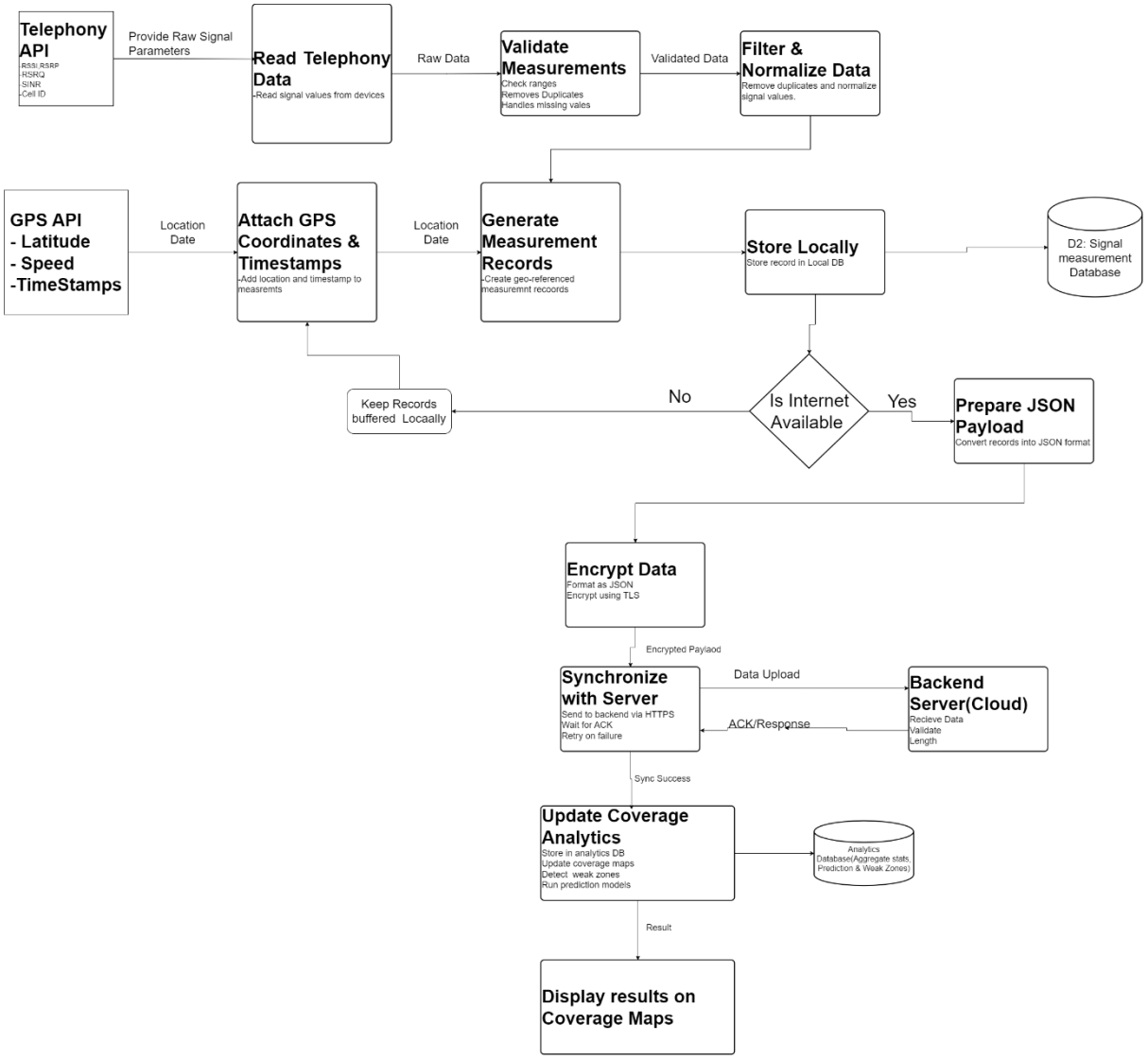

Figure 2.2: Level 1 Data Flow Diagram for Signal Data Collection

2.4 Importance of the Data Flow Diagram

The Data Flow Diagram is important because it provides a detailed understanding of how information moves within the system. It helps developers and stakeholders understand the relationship between processes, data stores, external entities, and system outputs. The DFD also simplifies system analysis, implementation planning, debugging, and maintenance activities.

For the Crowd sensed Drive Test Mobile Application, the DFD clearly demonstrates how crowd sensed measurements are collected from Android smartphones, processed securely through cloud infrastructure, analysed using prediction algorithms, and transformed into useful geographic coverage analytics.

3) Use Case Diagram

3.1 Foundations of Use Case Modeling

A Use Case Diagram is a behavioral UML (Unified Modeling Language) artifact that encapsulates the functional perimeter of a software platform. It maps out the external actors who interact with the platform and identifies the specific goals (use cases) they seek to achieve, without exposing underlying programmatic structures or code blocks. Within the software engineering lifecycle, this diagram serves as an operational blueprint derived directly from the Software Requirements Specification (SRS). It provides a concrete visualization of user boundaries, operational constraints, and cross-subsystem logic.

3.2 Definition of Key Artifacts

- **Actor:** An actor represents an external entity that plays a specific role when interacting with the system. Actors can be human users (e.g., end-users, administrators) or external hardware/software dependencies (e.g., cloud platforms, system APIs). They exist completely outside the system boundary.
- **Use Case:** A discrete piece of functionality within the system boundary that delivers a measurable, functional goal back to an actor.
- **System Boundary:** The structural encapsulation (rendered visually as a bounding box) that isolates the internal operations of the software system from external dependencies.

3.3 Architectural Actors and Subsystems

The system architecture decomposes external interfaces into distinct architectural boundaries based on execution scopes:

Primary Actors (Left Boundary)

- **Mobile User (Client Side):** The general consumer interacting with the mobile client. This actor establishes target permissions, instantiates operational telemetry loops, handles real-time maps, and synchronizes persistent buffers to the cloud repository.

Secondary Actors (Right Boundary)

- **System Administrator (Management Side):** The privileged administrative actor supervising platform performance. Responsible for user management, system diagnostic telemetry analysis, log evaluation, and generating cross-operator analytical views.
- **Backend Server:** The destination REST API layer executing critical computation. It evaluates uploaded telemetry structures, updates aggregated geographical views, and controls backend diagnostic auditing hooks.
- **GPS Services:** An underlying OS-level location infrastructure. Provides high-precision coordinates to anchor cellular signal traces with exact spatial values.
- **Telephony API:** The low-level Android system subsystem API. Exposes physical modem hardware variables to capture radio channel states dynamically.

3.4 Analysis of Diagram Relationships

The architectural diagram leverages standard UML stereotypes to manage code execution logic and enforce clear operational boundaries:

3.4.1 The <<include>> Stereotype (Mandatory Dependency)

An include relationship indicates that a base use case cannot complete its execution loop without explicitly running the target use case. It denotes unconditional behavior that is structurally embedded into the calling context.

Application Context*:* *The 'Start Monitoring' use case unconditionally triggers 'Collect Signal Data', 'Collect GPS Data', 'Grant Permissions', 'View Real-Time Signal', and 'Synchronize Data' as structurally included operations.*

3.4.2 The <<extend>> Stereotype (Conditional Flow)

An extend relationship is an exceptional, conditional, or optional operational flow. The extending use case only executes if specific logic criteria are satisfied at a predefined extension point within the base use case.

Application Context*: 'Receive Notifications' extends 'view logs' conditionally. Alerts are only fired when log structures reflect real-time network anomaly patterns or local coverage degradation*

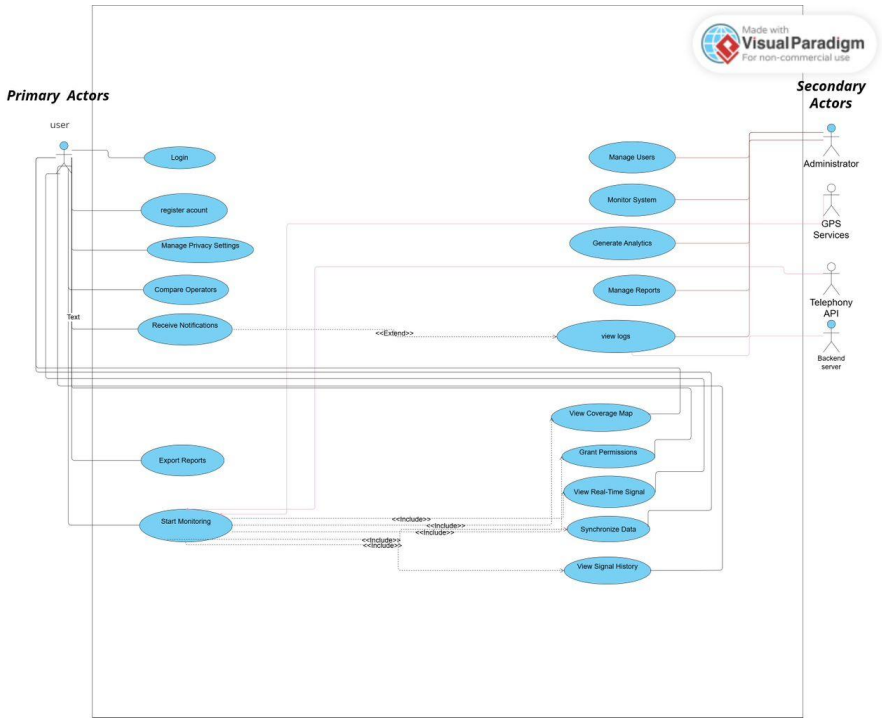

Figure 3: Use Case Diagram

3.5 Functional Use Case

The core system features map directly to these actors and dependencies, bridging the gap between mobile platform constraints and backend computing:

Use Case Identifier & Name	Primary Associated Actor	System Dependencies & Target Scope
Login / Register Account	Mobile User	Authenticates mobile sessions against secure tokens saved in the Android Keystore system.
Start Monitoring	Mobile User	Launches the persistent Android foreground monitoring service to capture cellular metrics.
View Coverage Map	Mobile User	Renders localized geo-referenced signal telemetry as interactive heatmap overlays.
Compare Operators	Mobile User	Evaluates signal quality across local regional networks (MTN, Orange, Camtel).
Manage Users / System	System Administrator	Performs user management, infrastructure monitoring, and database health audits.
Generate Analytics & Logs	System Administrator	Runs predictive mapping evaluations and processes historical data logs.

4) Sequence Diagram

4.1 Overview

The sequence diagram illustrates the chronological interaction between the various components of the application. It maps the flow of operations from the moment the user initiates the drive test until the data is successfully received by the backend for analysis.

4.2 Sequence of Operations

The process is structured into the following distinct operational phases:

Initialization Phase: The process begins when the user opens the mobile application and triggers the "Start Monitoring" command.

Data Collection Phase:

The application requests telephony data from the system.

The Telephony API returns crucial network parameters, including RSSI, RSRP, RSRQ, and SINR.

Simultaneously, the GPS module collects the current geo-location coordinates.

All collected parameters are combined into a single measurement record.

Storage Phase:

The record is stored locally in the SQLite database. This ensures data persistence when the device is offline.

Synchronization Phase:

The synchronization module checks for available internet connectivity.

If connectivity is confirmed, the data is encrypted and transmitted to the backend API via HTTPS.

The backend validates the received data and stores it in the coverage database.

The analytics engine then updates the coverage map, and the server returns an acknowledgment to the mobile application.

UI Update: Finally, the application updates the user interface to confirm the successful sync of the data.

4.3 Design Principles

Looping: The collection of data occurs in a continuous loop to capture network performance across the drive path.

Conditional Logic (Alt): The diagram utilizes an alternative (Alt) fragment to handle the condition of internet availability, ensuring that the app remains functional even when offline

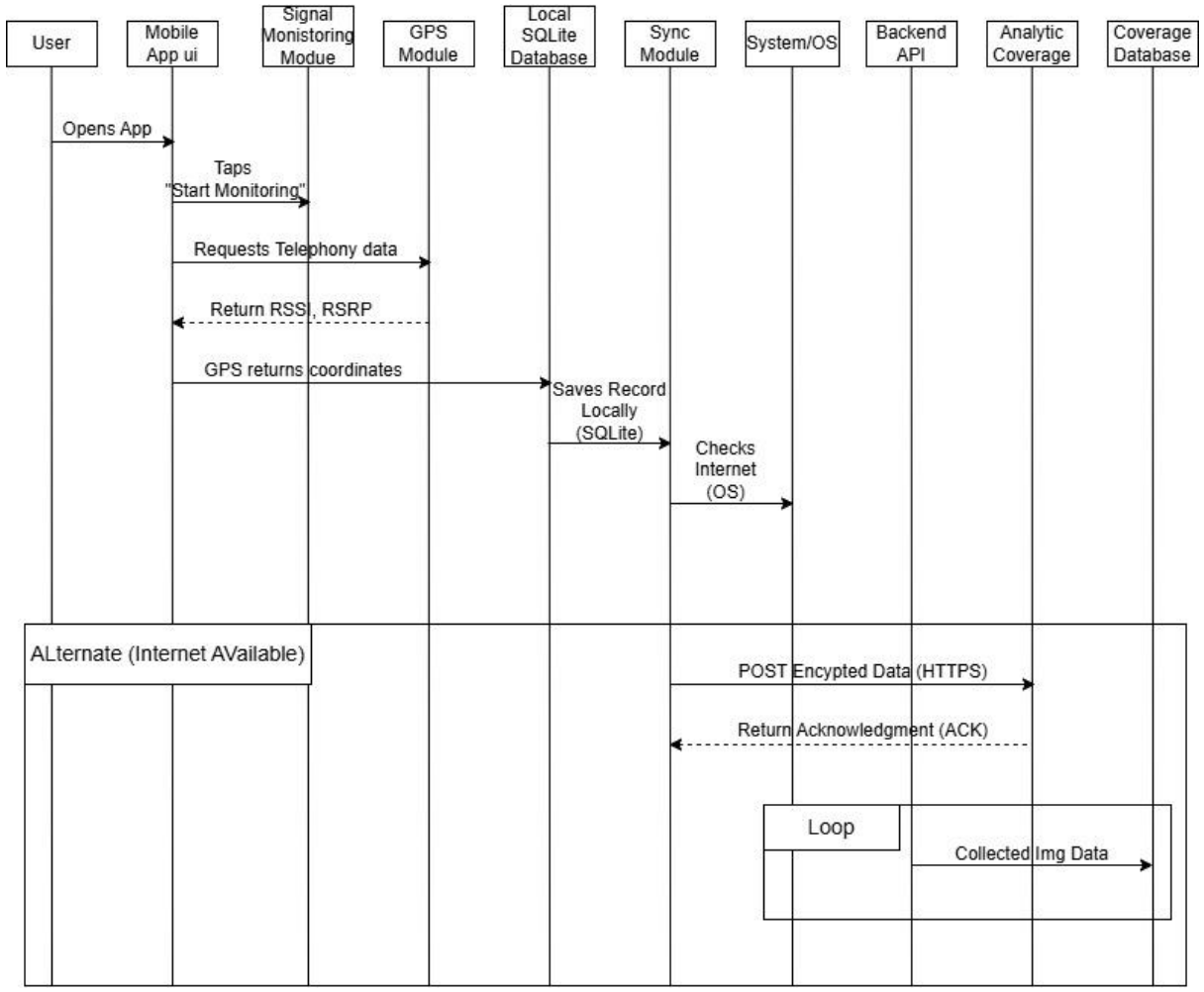

Figure 4: Sequence Diagram

5) Class Diagram

This report presents and explains the UML Class Diagram designed for the Crowd sensed Drive Test Mobile Application. The class diagram captures the structural blueprint of the application, defining ten (10) core classes, their attributes and methods, applicable Class, and the relationships between them.

5.1 System Overview

The application is organised into four logical packages:

- **User Management** : governs identity, authentication, and administrative control.
- **Monitoring & Measurement:** handles active signal capture and GPS positioning.
- **Coverage & Analytic:** processes and visualises the collected data.
- **Data & Infrastructure** : manages local persistence, synchronisation, and notifications.

5.2 Class Descriptions

CLASS	ATTRIBUTES	METHODS

User	– userId: String – username: String – email: String – passwordHash: String – deviceToken: String	+ register () + login (): Boolean + logout () + updateSettings ()
Admin	– adminId: String – role: String	+ generateReport (): Report + manageUsers () + monitorSystem ()
SignalMeasurement	– measurementId – RSSI / RSRP / RSRQ / SINR – CellID – networkType – timestamp	+ collectSignal () + validateMeasurement (): Boolean
GPSLocation	– latitude / longitude – altitude – speed – heading	+ getLocation (): GPSLocation + validateCoordinates (): boolean
MonitoringService	– monitoringStatus: Boolean – samplingInterval: Integer	+ startMonitoring () + stopMonitoring () + pauseMonitoring ()
LocalStorageManager	– databaseName: String – storageLimit: Integer	+ saveMeasurement () + retrievePendingData (): List + deleteSyncedData ()
SynchronizationManager	– syncStatus: String – retryCount: Integer	+ syncData () + encryptPayload (): String + retrySync ()
CoverageMap	– mapId: String – coverageZones: List – heatmapData: Map	+ generateMap () + renderHeatmap ()
AnalyticsEngine	– predictionModel: Model – analyticsData: Dataset	+ analyzeCoverage (): Report + detectCoverageHole (): List + generatePredictions ()
NotificationManager	– notificationId: String – message: String	+ sendAlert () + pushNotification ()

5.2 Relationships

Inheritance :Admin extends User

Admin inherits all attributes and methods of User (userId, login (), logout (), etc.) and adds its own administrative capabilities: generateReport (), manageUsers (), and monitorSystem (). This models the real-world scenario where an administrator is also a registered user of the system.

Association : User "1" to MonitoringService "1..*"

A single User is associated with one or more MonitoringService instances. This reflects the ability of a user to initiate and manage monitoring sessions across potentially multiple devices or sessions. The association is directed from User to MonitoringService.

Composition : MonitoringService "1" to SignalMeasurement "1..*"

MonitoringService composes one or more SignalMeasurement objects. Composition (filled diamond) implies a strong ownership: SignalMeasurement instances are created by and depend entirely on their parent MonitoringService. If the service is stopped and destroyed, its measurements are also destroyed. This accurately reflects that measurements only exist within an active monitoring session.

Aggregation: CoverageMap "1" to SignalMeasurement "0..*"

CoverageMap aggregates zero or more SignalMeasurement objects. Aggregation (hollow diamond) implies a weaker ownership: measurements may exist independently and be referenced by multiple coverage maps, e.g., historical data reused across different map versions. This supports the predictive coverage analysis use-case.

Dependency : SynchronizationManager to BackendAPI

A dashed dependency arrow indicates that SynchronizationManager relies on the BackendAPI at runtime to upload measurements. This is a loose coupling; the backend is an external system and not a first-class class within the application boundary. The «uses» stereotype makes this relationship explicit.

Additional Supporting Relationships

Several operational associations complete the data pipeline:

- MonitoringService uses GPSLocation to tag each measurement with its geographic position.
- MonitoringService delegates local persistence to LocalStorageManager (stores via).
- LocalStorageManager feeds pending data to SynchronizationManager for upload.
- SynchronizationManager triggers NotificationManager to alert the user on sync events.
- AnalyticsEngine reads CoverageMap data to run coverage-hole detection and predictions.
- Users receive push notifications from NotificationManager.

Design Observations

- Separation of Concerns : Each class has a focused responsibility, following the Single Responsibility Principle. Signal capture, storage, synchronisation, analytics, and notification are handled by separate classes.
- Offline-First Architecture: The presence of LocalStorageManager combined with SynchronizationManager with a retryCount attribute suggests the app is designed to function without a live network connection, queuing data until synchronisation is possible.
- Security Awareness: passwordHash (not plaintext) in User and encryptPayload () in SynchronizationManager indicate security-conscious design choices.

- Extensibility: The Admin-extends-User inheritance keeps the model DRY and allows the admin role to evolve independently of the base user model.
- Predictive Analytics Readiness : AnalyticsEngine includes a predictionModel attribute and generatePredictions () method, indicating the system is designed from the outset to support machine-learning-based coverage forecasting.

The class diagram provides a coherent and comprehensive structural model for the Crowd sensed Drive Test Mobile Application. The ten classes, collectively implement an end-to-end data pipeline from mobile signal capture to predictive coverage analysis. The use of composition, aggregation, inheritance, and dependency relationships accurately reflects the real-world lifecycle and ownership semantics of the system's data. The design prioritises offline resilience, security, and separation of concerns, forming a solid foundation for implementation.

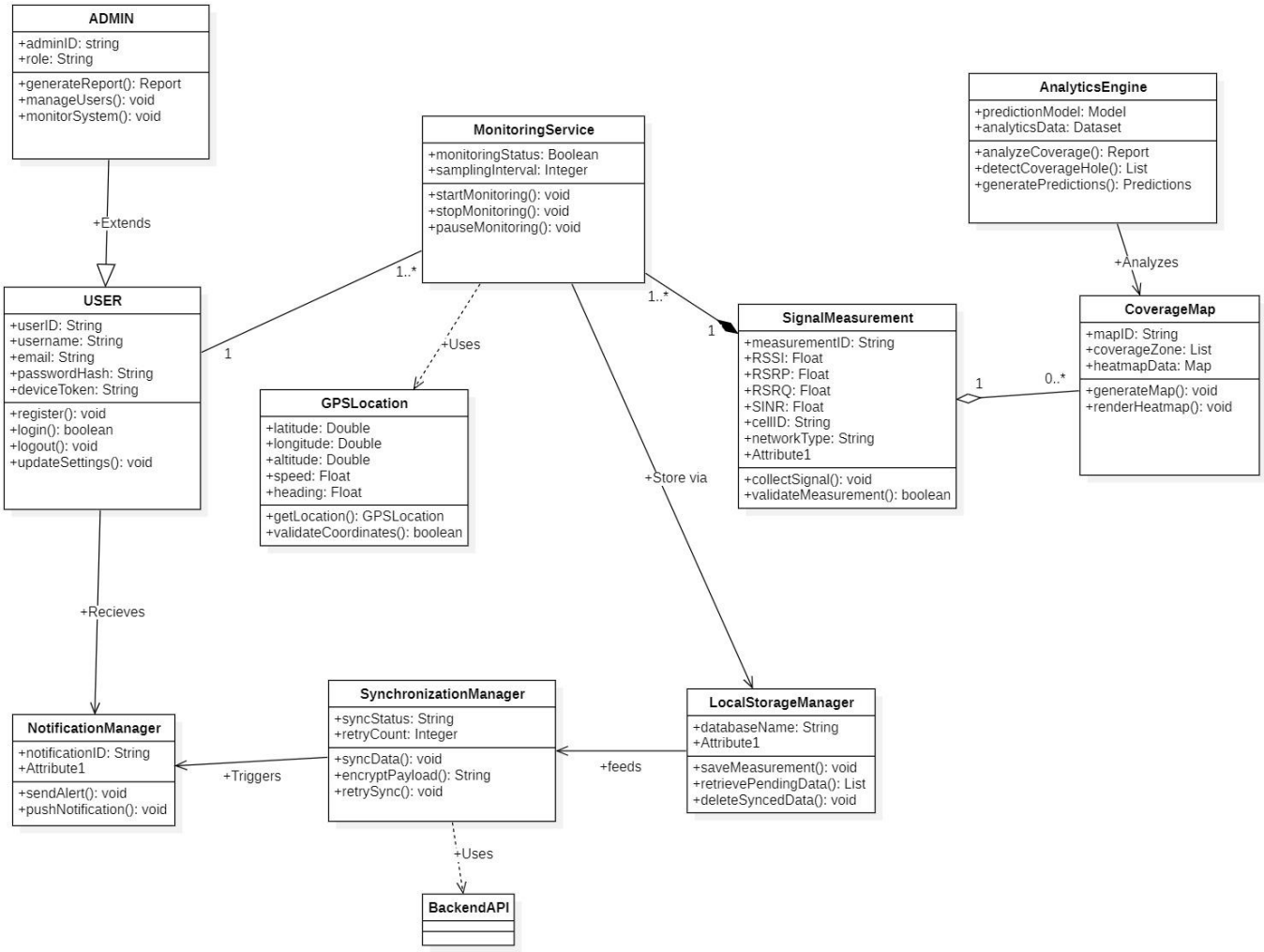

Figure 2.1: Class Diagram

6) Deployment Diagram

A Deployment Diagram shows how the software design turns into the actual physical system where the software will run. They show where software components are placed on hardware devices and shows how they connect with each other. This diagram helps visualize how the software will operate across different devices

6.1 Key elements of a Deployment Diagram

Below are the key elements of deployment diagram:

- **Nodes**: These represent the physical hardware entities where software components are deployed, such as servers, workstations, routers, etc.
- **Components**: Represent software modules or artifacts that are deployed onto nodes, including executable files, libraries, databases, and configuration files.
- **Artifacts**: Physical files that are placed on nodes represent the actual implementation of software components. These can include executable files, scripts, databases, and more.
- **Dependencies**: These show the relationships or connections between nodes and components, highlighting communication paths, deployment constraints, and other dependencies.
- **Associations**: Show relationships between nodes and components, signifying that a component is deployed on a particular node, thus mapping software components to physical nodes.
- **Deployment Specification**: This outlines the setup and characteristics of nodes and components, including hardware specifications, software settings, and communication protocols.

Communication Paths: Represent channels or connections facilitating communication between nodes and components and includes network connections, communication protocols.

The Deployment Diagram of the Crowd sensed Drive Test Mobile Application illustrates the physical architecture of the system and shows how software components are deployed across different hardware devices and cloud infrastructure. The diagram explains how Android smartphones, backend servers, database servers, administrator workstations, and external geographic services communicate and work together to support crowd sensed mobile network coverage prediction.

The deployment process begins from the Android smartphone, which acts as the client device used by end-users. The smartphone contains the mobile application, monitoring service, synchronization module, and SQLite local database. The application collects mobile network measurements such as RSSI, RSRP, RSRQ, SINR, and GPS coordinates from the user's environment.

The collected measurements are temporarily stored inside the SQLite database and later synchronized securely with the Cloud Backend Server using HTTPS, RESTful APIs, JSON data exchange, and TLS 1.2 encryption. The backend server contains important services such as the REST API, Authentication Service, Synchronization Service, and Analytics Engine. These services process uploaded measurements, manage user authentication, and generate coverage prediction analytics.

The backend server communicates with the Database Server through SQL queries to store user accounts, crowd sensed measurements, synchronization logs, and analytics results. The system also integrates with the Google Maps Server to provide geographic visualization, map rendering, and coverage heatmaps.

Finally, the Administrator Workstation connects securely to the backend server through a monitoring dashboard where administrators can analyze coverage statistics, monitor synchronization activities, and generate reports. The deployment diagram therefore provides a clear representation of how the different physical and software components of the system interact to support real-time crowd sensed mobile network coverage analysis and prediction.

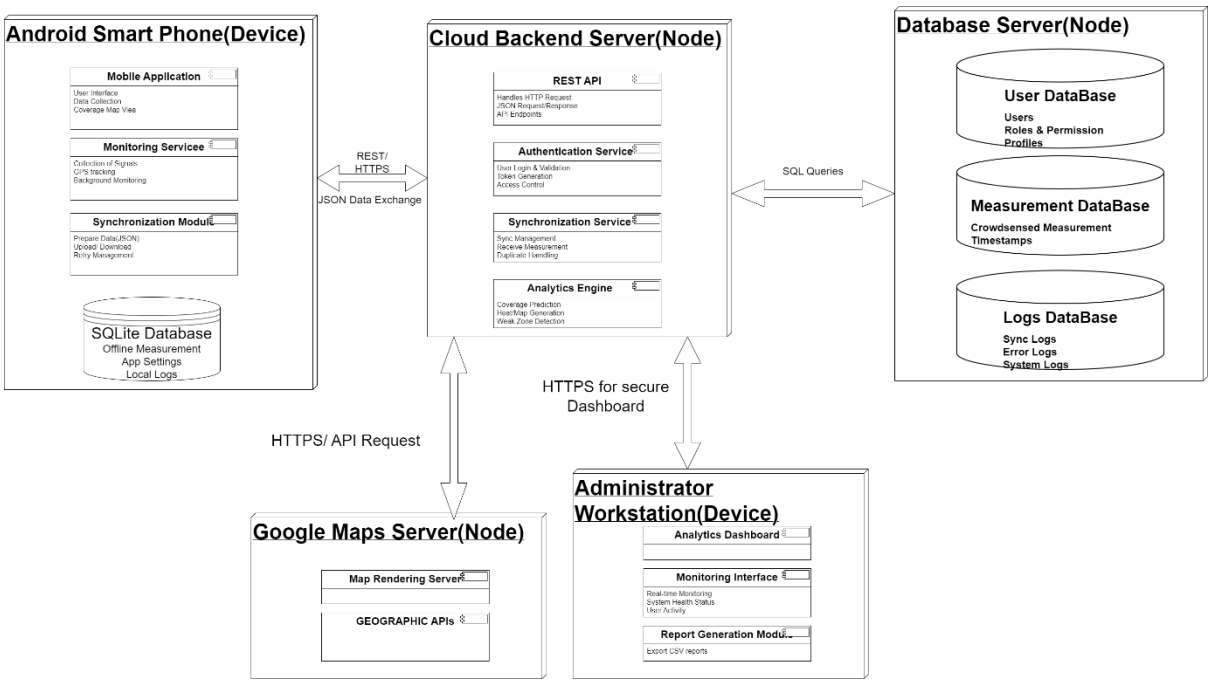

Figure 5: Deployment Diagram

7) Conclusion

The system modelling and design phase provided a comprehensive representation of the architecture, structure, communication flow, and operational behaviour of the Crowd sensed Drive Test Mobile Application for Coverage Prediction. Through the development of modelling diagrams such as the Context Diagram, Data Flow Diagram (DFD), Use Case Diagram, Sequence Diagram, Class Diagram, and Deployment Diagram, the major functional and structural aspects of the system were clearly defined and analysed.

The modelling phase made it possible to understand how crowd sensed mobile network measurements are collected from Android smartphones, processed through cloud infrastructure, synchronized securely, stored in databases, analysed using coverage prediction algorithms, and visualized geographically using mapping services. The diagrams also helped identify the relationships between users, system components, databases, APIs, and cloud services involved in the application architecture.

In addition, the system modelling process established a clear foundation for the development lifecycle of the application. It simplified the understanding of system requirements, improved communication among development team members, and provided guidance for implementation, testing, deployment, and future maintenance of the platform.

With the completion of the system modelling and design phase, the project now transitions into the User Interface (UI) Design and System Implementation phase. During this next stage, the architectural designs and workflows represented in the diagrams will be transformed into a functional Android application and backend infrastructure. The UI design phase will focus on creating user-friendly interfaces, interactive dashboards, monitoring screens, analytics views, and report generation modules that provide a smooth user experience. The implementation phase will involve the development of the Android application, cloud backend services, RESTful APIs, local SQLite storage, synchronization services, analytics engine, and integration with Google Maps APIs.

This transition from modelling to implementation marks an important step toward building a complete crowd sensed mobile network coverage prediction system capable of real-time monitoring, secure synchronization, geographic visualization, and intelligent coverage analytics.

8) References

1. Pearson – Software Engineering by Ian Sommerville – Discusses software engineering principles, system analysis, and system modeling techniques.
2. Software Engineering Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education.
3. Systems Analysis and Design Dennis, A., Wixom, B., & Roth, R. (2015). *Systems Analysis and Design*. Wiley Publishing.
4. Software Requirements Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd Edition). Microsoft Press.
5. Requirements Engineering Sommerville, I., Lock, R., & Storer, T. (2012). “Information Requirements for Enterprise Systems.”